Database Index Changes

Requirement 3 (Performance & Caching) · Branch `refactor/products-caching-decorator` · 12 November 2026

Summary. Five indexes were added and one was removed, across three modules. Each index matches a query the API actually runs. With realistic data (200,000 products, 300,000 orders, 600,000 order items, 50,000 users), the affected queries dropped from **4–62 ms to under 0.25 ms**, because Postgres now reads the requested page straight from an index instead of scanning and sorting the whole table. All changes are EF Core migrations, applied to the local database and tested both ways (up and down).

1. What changed

Table	Index	Change	Serves
`products.Products`	`IX_Products_CreatedAt_Id` (`CreatedAt`, `Id`)	Added	Product list page, sorted by creation time
`products.Products`	`IX_Products_LowerCategory_CreatedAt_Id` (`lower(Category)`, `CreatedAt`, `Id`)	Added	Product list filtered by category, and its count
`orders.Orders`	`IX_Orders_UserId_CreatedAt_Id` (`UserId`, `CreatedAt`, `Id`)	Added	A customer's orders, newest first
`orders.Orders`	`IX_Orders_CreatedAt_Id` (`CreatedAt`, `Id`)	Added	Admin "all orders", newest first
`orders.Orders`	`IX_Orders_UserId` (`UserId`)	Removed	Fully covered by `IX_Orders_UserId_CreatedAt_Id`
`users.Users`	`IX_Users_Email` (`Email`), **unique**	Added	Login lookup; blocks duplicate accounts
`orders.OrderItems`	`IX_OrderItems_OrderId`	Kept	Loading an order's items — already correct
`inventory.*`	primary keys	Kept	Every inventory-service query already uses its primary key

2. Results

Measured with `EXPLAIN ANALYZE` on the queries EF Core generates, with and without the indexes.

Query	Before	After	Plan change
Product list, page 1 (20 rows)	61.7 ms	**0.05 ms**	Full scan + sort of 200k rows → index scan, stops after 20 rows
Product list by category, page 1	27.3 ms	**0.04 ms**	Full scan with filter + sort → index scan on the category expression
Category count (for `TotalPages`)	24.7 ms	**3–12 ms**	Full scan → bitmap index scan of ~10k matching rows
Customer's orders page + items	0.61 ms	**0.11–0.17 ms**	Index lookup + sort → index scan already in order (backward)
Admin all-orders page + items	22.2 ms	**0.13–0.22 ms**	Full scan + sort of 300k rows → index scan (backward)
Login by email	4.1 ms	**0.03 ms**	Full scan of 50k users → unique index lookup

Ranges show two separate runs. The second "after" run was against the indexes created by the real migrations.

3. Each index explained

3.1 `IX_Products_CreatedAt_Id` — product list order

Query it serves (`ProductRepository.GetPageAsync`):

```
SELECT * FROM products."Products" ORDER BY "CreatedAt", "Id" LIMIT 20 OFFSET 0;
```

> A B-tree index stores rows already sorted by its columns. Because the index columns match the `ORDER BY` exactly, Postgres walks the index from the start and stops after 20 entries. Without it, it must read all 200,000 rows and sort them just to return 20. `Id` is included because the query sorts by it as a tie-breaker (it keeps pages stable when two products share a timestamp).

Declared in `ProductsDbContext`:

```
builder.HasIndex(p => new { p.CreatedAt, p.Id }).HasDatabaseName("IX_Products_CreatedAt_Id");
```

3.2 `IX_Products_LowerCategory_CreatedAt_Id` — category filter

Query it serves:

```
SELECT * FROM products."Products"
WHERE lower("Category") = 'electronics'
ORDER BY "CreatedAt", "Id" LIMIT 20;

SELECT count(*) FROM products."Products" WHERE lower("Category") = 'electronics';
```

> The C# code compares `p.Category.ToLower() == normalized`, which EF translates to `lower("Category") = ...`. Postgres can only use an index when the query's expression matches the indexed one, so a plain index on `Category` would never be used. This is an **expression index** on `lower("Category")`, followed by the sort columns, so one index handles both the filter and the order.

Declared in the migration with raw SQL, because EF Core's `HasIndex` cannot express an index on a function:

```
migrationBuilder.Sql(
    """CREATE INDEX "IX_Products_LowerCategory_CreatedAt_Id" ON products."Products"
(lower("Category"), "CreatedAt", "Id");""");
// Down():
migrationBuilder.Sql("""DROP INDEX IF EXISTS products."IX_Products_LowerCategory_CreatedAt_Id";""");
```

3.3 `IX_Orders_UserId_CreatedAt_Id` — a customer's orders (replaces `IX_Orders_UserId`)

Query it serves (`OrderService.GetOrdersAsync` for a customer):

```
SELECT … FROM orders."Orders" WHERE "UserId" = @id
ORDER BY "CreatedAt" DESC, "Id" DESC LIMIT 21 OFFSET 0;   -- + join to OrderItems
```

> The old index on `UserId` alone found the customer's orders but then had to sort them. With `CreatedAt, Id` added after `UserId`, the matching entries are already in date order, so Postgres reads the first 21 and stops. The index columns are ascending; Postgres simply reads the index backwards for `DESC` ("Index Scan Backward" in the plan), so no special descending index is needed.

> **Why the old index was removed:** an index on (A, B, C) can also answer queries on A alone. EXPLAIN confirmed Postgres uses the new index for plain `UserId` lookups, so `IX_Orders_UserId` became dead weight that only slowed down every order insert.

Declared in `OrdersDbContext` (replacing `builder.HasIndex(o => o.UserId)`):

```
builder.HasIndex(o => new { o.UserId, o.CreatedAt, o.Id
}).HasDatabaseName("IX_Orders_UserId_CreatedAt_Id");
```

3.4 `IX_Orders_CreatedAt_Id` — admin "all orders"

Query it serves (`GetOrdersAsync` for an Admin, no user filter):

```
SELECT … FROM orders."Orders" ORDER BY "CreatedAt" DESC, "Id" DESC LIMIT 21 OFFSET 0;
```

> Same reasoning as 3.1: the admin list has no filter, only an order. Without this index Postgres scanned and sorted all 300,000 orders for every page.

```
builder.HasIndex(o => new { o.CreatedAt, o.Id }).HasDatabaseName("IX_Orders_CreatedAt_Id");
```

3.5 `IX_Users_Email` (unique) — login and duplicate protection

Queries it serves (`UserRepository.GetByEmailAsync` / `ExistsByEmailAsync`):

```
SELECT … FROM users."Users" WHERE "Email" = @email;
```

> **Speed:** every login used to scan the whole Users table.
> **Correctness:** registration checks "does this email exist?" and then inserts. Two registrations arriving at the same moment could both pass the check and create two accounts with the same email. A **unique** index makes the database itself reject the second insert, so the race cannot produce duplicates. The existing data was checked for duplicates first (none found).

```
modelBuilder.Entity<User>().HasIndex(u => u.Email).IsUnique().HasDatabaseName("IX_Users_Email");
```

4. Files changed

File	Change
`src/Modules/Products/Persistence/ProductsDbContext.cs`	Declares `IX_Products_CreatedAt_Id`; comment pointing to the raw-SQL index
`src/Modules/Products/Migrations/20261112061240_AddProductListIndexes.cs`	New migration: both product indexes (one via `migrationBuilder.Sql`)
`src/Modules/Orders/Persistence/OrdersDbContext.cs`	Replaces the `UserId` index with the two composite indexes
`src/Modules/Orders/Migrations/20261112061245_AddOrderListIndexes.cs`	New migration: drops `IX_Orders_UserId`, creates the two order indexes
`src/Modules/Users/Persistence/UserDBContext.cs`	Declares the unique `Email` index
`src/Modules/Users/Migrations/20261112061249_AddUniqueUserEmailIndex.cs`	New migration: creates `IX_Users_Email`
`*.Designer.cs`, `*ModelSnapshot.cs`	Generated by EF Core alongside each migration

Applying on another machine

```
dotnet ef database update --project src/Modules/Products --startup-project src/Host --context
ProductsDbContext
dotnet ef database update --project src/Modules/Orders   --startup-project src/Host --context
OrdersDbContext
dotnet ef database update --project src/Modules/Users    --startup-project src/Host --context
UsersDbContext
```

5. How it was verified

- **Realistic data, safely:** 200k products, 300k orders, 600k items and 50k users were inserted inside a single database transaction, measured with `EXPLAIN ANALYZE`, and then rolled back. The real data was unchanged afterwards (6 products, 4 orders, 9 users).
- **Migrations both ways:** the Products migration (which contains hand-written SQL) was reverted and re-applied; the index list was checked after each step.
- **Plans after migration:** every list query uses its new index (see section 2), including plain `UserId` lookups on the new composite index.
- **API smoke test:** login 200; registering an existing email returns 409 "Email is already registered."; product list, category list and admin orders list all return 200 with the expected rows.

6. Notes and follow-ups

Email is case-sensitive. `A@x.com` and `a@x.com` are still treated as different users, so the unique index allows both. To make emails case-insensitive, lowercase them on register and login, or index `lower("Email")` instead.

Registration race now returns 500, not a duplicate. In the rare case two identical registrations race, the second insert hits the unique index and the API currently answers 500. The data stays correct; mapping that database error to 409 would make the response correct too.

Deep pages are still linear. Page 5,000 (`OFFSET 100000`) takes ~12–18 ms even with the index, because Postgres still steps over the skipped rows. Keyset paging (`WHERE ("CreatedAt","Id") > (last seen) LIMIT 20`) makes every page as fast as page 1 if very deep pages matter.

Connection pool vs. Postgres limit. While this work started, Postgres was refusing new connections ("too many clients"): the running API held almost all of Postgres's 100 connections, very likely after a heavy JMeter run. Npgsql's default pool size is also 100, so one API instance can exhaust the server. Setting `Maximum Pool Size` (for example 50) in the connection string, or raising `max_connections`, avoids this — especially once Requirement 5 runs several API instances.